ICS 2315: Data Architecture & Warehousing

Coursework Implementation & Analytical Summary Report

Student: Kelvin Abidha
Module: NoSQL & Graph Analytics
Date: October 2026

GitHub Repository

81,504	6.4x	5.36 ms	262 / 362
NSE RECORDS CLEANED	MONGO INDEX SPEEDUP	CASSANDRA PARTITION READ	GRAPH NODES / LINKS

1. Executive Summary & Theoretical Framework

This deliverable documents the end-to-end design, implementation, and empirical evaluation of three non-relational database architectures across two distinct enterprise datasets:

- **Document Store (MongoDB)**: Document modeling, B-Tree compound indexing benchmarks, execution plan diagnostics (`.explain()`), and aggregation pipelines.
- **Distributed Wide-Column Store (Apache Cassandra)**: Query-driven data modeling (Chebotko methodology), token ring consistent hashing, and unpartitioned scan rejection.
- **Graph Database (Neo4j)**: Index-free adjacency, topological entity-relationship modeling, and Cypher multi-hop traversals on road safety records.

> **Core Textbook Alignment (Database System Concepts 7th Ed, Chapter 10)**
>
> This project demonstrates **Consistent Hashing** (p. 467) via Cassandra's Murmur3Partitioner, **Query Optimization & B-Tree Indexing** (Chapters 12–13) via MongoDB explain plans, **MapReduce Aggregations** (p. 483) via MongoDB aggregation pipelines, and **Index-Free Adjacency** in Neo4j where link traversals execute at raw memory pointer speeds ($O(1)$ per hop).

2. Assignment 1: NoSQL Data Processing (NSE Stock Market)

2.1 Data Engineering & ETL Pipeline (src/etl.py)

Six yearly datasets (2007–2012) containing 81,504 raw market trades were consolidated. Key transformations implemented:

- **Header Schema Normalization**: Resolved an integration anomaly where 2008 used Title Case (`Date` , `Adjusted`) while 2007/2009–2012 used uppercase (`DATE` , `Adjust`).
- **Numeric Coercion**: Removed formatting commas from volume (`"7,800"` → `7800`) and stripped `%` characters from daily returns.
- **Missing Value Standardization**: Replaced non-standard dash placeholders (`"-"`) with IEEE `NaN` / BSON `null` .
- **Chronological Formatting**: Standardized date strings into native ISO timestamps (`datetime64[us]`).

2.2 MongoDB Implementation & Indexing Experiment (src/mongo_manager.py)

Records were modeled into nested BSON documents for optimal **locality of reference**:

```
{
  "code": "EQTY",
  "name": "Equity Bank",
  "date": ISODate("2007-01-02T00:00:00Z"),
  "trading": { "day_price": 141.0, "volume": 7500, "change_pct": 2.13 },
  "intraday": { "low": 140.0, "high": 145.0 },
  "yearly_range": { "low": 19.5, "high": 36.5 }
}
```

The Indexing Experiment: Querying Safaricom (`'SCOM'`) in 2012 with `volume > 1,000,000` , sorted by `date DESC` :

INDEXING SCENARIO	WINNING STAGE	DOCS EXAMINED	KEYS EXAMINED	RETURNED DOCS	EXECUTION LATENCY	SPEEDUP
Scenario 1: No Index	SORT → COLLSCAN	81,504	0	220	83 ms	1.0x (Baseline)
Scenario 2: Single Index `{'code': 1}`	SORT → FETCH	1,146	1,146	220	21 ms	~4.0x
Scenario 3: Compound `{'code': 1, 'date': -1}`	FETCH → IXSCAN	251	251	220	**13 ms**	**~6.4x**

> **Database Theory: The ESR Rule (Equality → Sort → Range)**
>
> In Scenario 1, MongoDB scanned all 81,504 records into RAM and executed an in-memory sort buffer. In Scenario 3, by ordering the compound index with Equality (`code`) first, followed by Sort (`date DESC`), MongoDB read the index tree in sorted sequence—completely eliminating the in-memory sort and dropping examined documents to 251.

2.3 Apache Cassandra Distributed Modeling (src/cassandra_manager.py)

In distributed wide-column architecture, tables are designed strictly around target queries:

```
-- Historical Stock Lookups by Ticker
CREATE TABLE coursework_nse.stock_history_by_ticker (
    code text,
    date date,
    name text,
    day_price double,
    volume bigint,
    change_pct double,
    PRIMARY KEY ((code), date)
) WITH CLUSTERING ORDER BY (date DESC);

-- Monthly Market-Wide Snapshots
CREATE TABLE coursework_nse.daily_market_snapshot (
```

```
    year int,
    month int,
    date date,
    code text,
    name text,
    day_price double,
    volume bigint,
    PRIMARY KEY ((year, month), date, code)
) WITH CLUSTERING ORDER BY (date DESC, code ASC);
```

> **Architectural Experiment: Scan Rejection (code=2200)**
>
> Attempting `SELECT * FROM stock_history_by_ticker WHERE day_price > 100` triggered: `InvalidRequest: code=2200 [Invalid query] message="Cannot execute this query as it might involve data filtering and thus may have unpredictable performance... use ALLOW FILTERING"` . Cassandra forbids unpartitioned full scans by default because broadcasting queries to all nodes breaks linear scalability. Forcing `ALLOW FILTERING` degraded latency to 31.51 ms.

3. Assignment 2: Graph Data Analytics (Traffic Incidents)

Constructed in Neo4j using 78 geocoded crash records (375 fatalities), yielding **262 Nodes** and **362 Relationships**.

3.1 Relationship Severity Parameter Rating

Rated severity scores directly onto `[:OCCURRED_ON]` relationships: `Severity Score = (total_dead × 2.5) + spot_freq + risk_weight`

SEVERITY PARAMETER LEVEL	RATED LINKS	FATALITIES	AVG SEVERITY SCORE	CLASSIFICATION CRITERIA
Severe	56 links	362 deaths	20.54	≥ 3 deaths or High risk classification
Moderate	13 links	13 deaths	5.42	1–2 deaths or Medium risk classification
Minor	9 links	0 deaths	2.11	0 deaths or Low risk classification

3.2 Cypher Analytical Insights

- **Deadliest Corridors**: *Mombasa–Nairobi Highway* (65 fatalities, 5 Severe), *Nakuru–Kericho Highway* (58 fatalities, 3 Severe), and *Nakuru–Nairobi Highway* (53 fatalities, 7 Severe).
- **Multi-Hop Root Cause Traversal**: Tracing paths from Nakuru County revealed heavy freight brake failures (52 deaths at Londiani) and public service vehicle (PSV) head-on collisions (32 deaths at Kayole Bridge & Naivasha).
- **Blackspot Centrality**: *Nithi Bridge* (4 collisions, 17 fatalities) and *Londiani Junction* (2 collisions, 55 fatalities) exhibit highest degree centrality.

4. Visual Graph Evidence (Neo4j Browser UI)

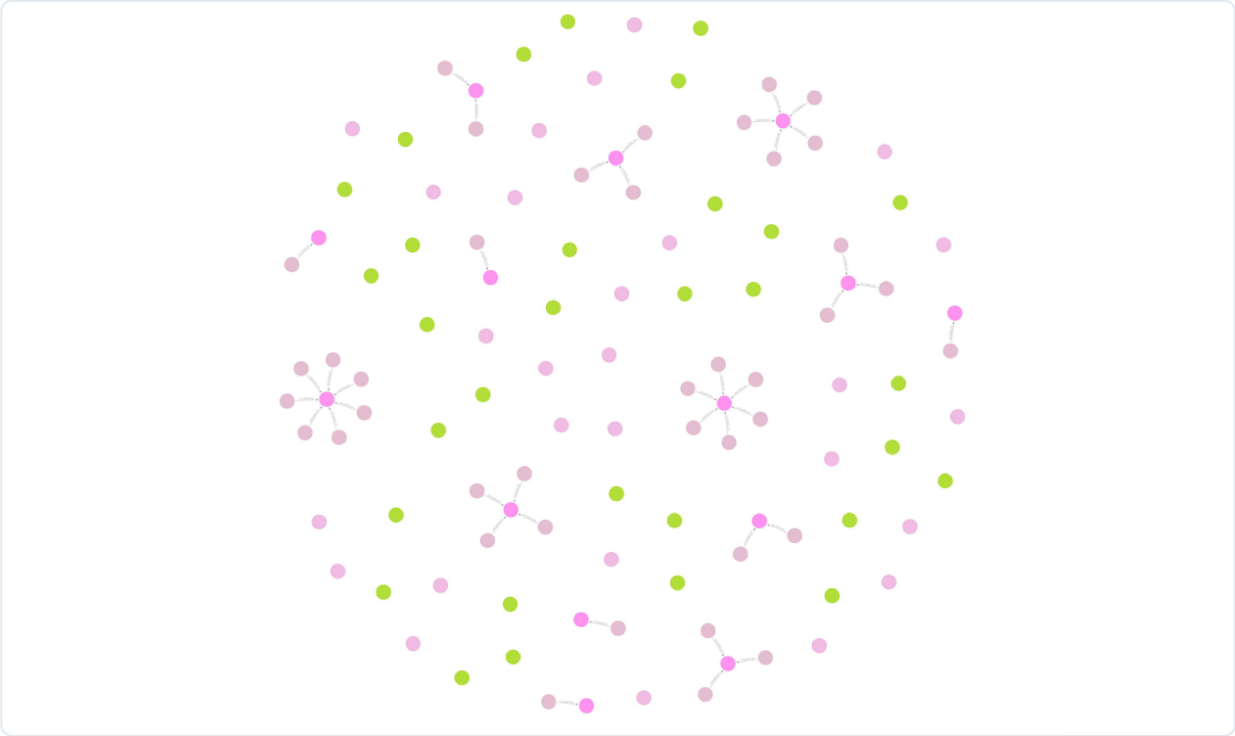


Figure 1: Neo4j Graph Visualization of High-Severity Road Accidents, Blackspots, and Root Causes across Kenya.

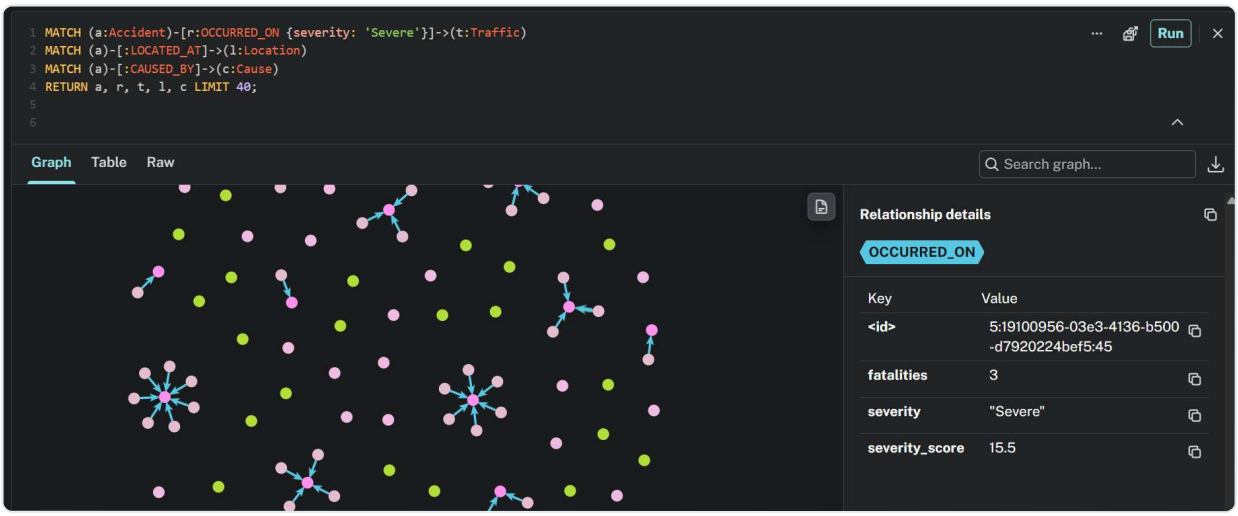


Figure 2: Edge Property Inspector demonstrating severity parameter rating (`severity: "Severe"` , `fatalities: 3` , `severity_score: 15.5`) on the `OCCURRED_ON` relationship.

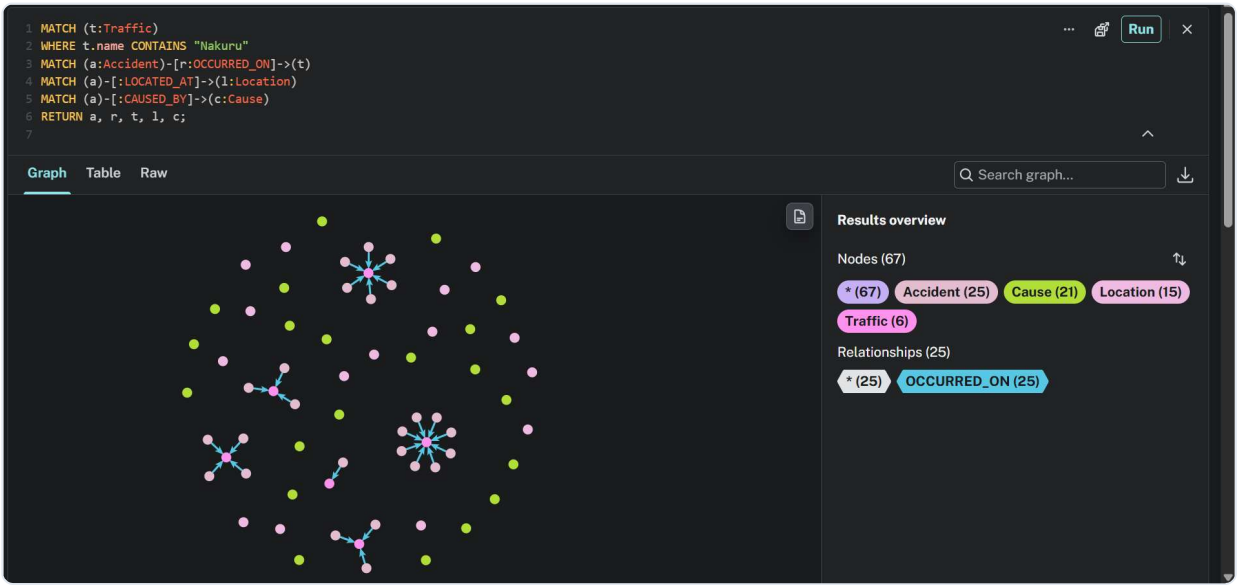


Figure 3: Subgraph analysis of the Nakuru Transit Corridor revealing incident clustering and primary vehicular collision causes.

5. Comparative Architectural Critique

DIMENSION	RELATIONAL (RDBMS)	DOCUMENT STORE (MONGODB)	WIDE-COLUMN (CASSANDRA)	GRAPH DATABASE (NEO4J)
Core Data Model	2D Tables & Tuples	Nested BSON Documents	Sparse Wide-Columns	Nodes & Relationships
Schema Design	Normalized 3NF	Hierarchical / Embedded	Query-Driven Denormalization	Property Graph Topology
Partitioning Mechanism	Range / Hash Sharding	Hash / Range via `mongos`	Consistent Hashing Ring	Clustered / Fabric Sharding
Query Mechanism	Declarative SQL (JOINs)	MQL + Aggregation Pipeline	CQL (No JOINs allowed)	Cypher (Pattern Matching)
Performance Scaling	Vertical (Scale-Up)	Horizontal (Read Replicas)	Masterless Linear Write	Index-Free Adjacency (O(1))

6. Verification & Reproduction

All scripts and schemas are maintained in the version-controlled project repository: https://github.com/KelvinAbidha/Data-Warehousing-Project.

```
# 1. Start Cassandra & Neo4j containers
docker compose up -d
```

```
# 2. Run ETL pipeline for 81,504 market records
python src/etl.py

# 3. Execute MongoDB ingestion, indexing benchmarks & aggregations
python src/mongo_manager.py

# 4. Execute Cassandra keyspace creation & query modeling
python src/cassandra_manager.py

# 5. Execute Neo4j Property Graph construction & Cypher analytics
python src/graph_manager.py
```